

Yusuf ATMACA

✉ yusufatmaca@protonmail.com 🔗 yusufatmaca.github.io in yusufatmaca97 🌐 yusufatmaca

Education

İzmir Institute of Technology

Oct 2019 – June 2025

BSc in Computer Engineering

- **Elective Courses I took:** Heterogeneous Parallel Programming, Real-Time Systems, Parallel Programming Patterns, Deep Learning for NLP, Compiler Design and Construction, Optimization Methods

Experience

Software Engineer

İzmir 35410, TR

ete technology®

Aug 2025 – Present

- Contributing to the development of simulation and modeling tools using C/C++, with an emphasis on object-oriented design and efficient data structures.
- Working on GPU-accelerated components using CUDA and parallel programming techniques to improve computational performance.
- Collaborating in a multidisciplinary team to build real-time and scalable software systems.
- Applying mathematical and algorithmic approaches in performance-critical systems relevant to simulation environments.

System Engineer Long-Term Intern

Kocaeli 41480, TR

Yapi Kredi Technology

Feb 2025– June 2025

- Assisted in patch management and vulnerability tracking for Windows OS.
- Documented system activities and shared them with operations teams when necessary.
- Collaborated closely with security and software teams to ensure system reliability and continuity.
- Managed and tracked internal requests through a ticket-based platform, taking action on incoming records.

Back-End Developer Intern

İzmir 35430, TR

DELTA Smart Technologies

June 2024 – Oct 2024

- Developed an internal company organization system using Java Spring Boot and MySQL.
- Designed and implemented RESTful APIs to manage user roles, departments, and team hierarchies.
- Followed clean code principles and contributed to scalable backend architecture.
- Collaborated in an agile environment and participated in code reviews and debugging sessions.
- Gained hands-on experience with relational database modeling, data validation, and deployment practices.

Engineer Intern

İzmir 35210, TR

T.C. Sanayi ve Teknoloji Bakanlığı

Aug 2024 – Sep 2024

- Completed a mandatory engineering internship as part of undergraduate program requirements.
- Gained exposure to government-led industrial operations and observed administrative workflows within a public-sector engineering context.

Projects

Bachelor's Thesis – Louvain Algorithm with OpenMP GPU Offloading

[louvain-openmp-gpu](#) 

- Implemented the Louvain community detection algorithm using OpenMP GPU offloading as part of a bachelor's thesis project.
- Closely followed the approach presented in [Community Detection on GPUs: A Comparative Approach](#) by Naim et al. to ensure methodological consistency.
- Focused on modularity optimization and vertex bucketing strategies to achieve parallel performance on heterogeneous systems.
- Gained hands-on experience with shared-memory parallelism, graph-based algorithm design, and performance evaluation across GPU frameworks.

lazybones - kernel

[lazybones-kernel](#) 

- Developed a minimal educational operating system kernel from scratch as part of a systems programming course, exploring the boot process, low-level hardware interaction, and disk image manipulation.
- Implemented an experimental task management layer demonstrating basic scheduling and context switching, with support for configurable stack size, period, and priority attributes through a unified task creation interface.
- Designed a real-time clock module that retrieves the current time from the CMOS RTC and updates via timer interrupts, eliminating busy-waiting and ensuring smooth progression with a persistent zero-padded 24-hour display.
- Built a minimal command-line shell supporting essential commands such as `help`, `settime`, and `reboot`, with argument validation, error handling, and command parsing for robust and predictable behavior.
- Gained hands-on experience with bare-metal programming, bootloaders, interrupt handling, and bootable floppy disk image preparation.

lazybones - shell

[lazybones-shell](#) 

- Designed and implemented a minimal Unix-like shell from scratch in modern C++, inspired by Stephen Brennan's shell tutorial and following POSIX shell conventions.
- Implemented core shell functionality including command parsing, process creation via `fork/exec`, input/output redirection, and pipelines for chaining commands.
- Developed a set of built-in commands alongside external program execution, with a clean separation between the parsing, evaluation, and execution stages of the shell pipeline.
- Applied modern C++ features such as RAII, smart pointers, and the standard template library to write clean, memory-safe, and maintainable code.
- Strengthened understanding of process management, system calls, signal handling, and the inner workings of Unix shells through practical implementation.

Parallelizing NFA Matching

[C-NFA-3150](#) 

- Contributed to an existing Non-Deterministic Finite Automata (NFA) matching project by parallelizing a previously serial implementation.
- Applied parallel programming principles to accelerate pattern matching, leveraging techniques such as data decomposition and thread-level parallelism.
- Improved overall performance and scalability while preserving the correctness of the original algorithm.
- Gained practical experience in transforming sequential code into efficient parallel executions using real-world automata theory applications.

Estimating π Using Monte Carlo Simulation with CUDA

[estimate-pi-using-monte-carlo-method](#) 

- Implemented a parallel Monte Carlo simulation to estimate the value of π using NVIDIA's CUDA programming model.
- Offloaded the random point generation and circle-hit detection to the GPU, enabling massive parallelism for high-speed computation.
- Demonstrated significant performance improvement over CPU-based implementations, highlighting CUDA's suitability for embarrassingly parallel problems.
- Developed a clear understanding of CUDA kernels, memory management (global vs. shared), and thread hierarchy (grids and blocks).
- Documented the algorithm in detail in the README file.

Certificates

- | | |
|--|------------|
| ◦ CUDA | NVIDIA DLI |
| – Accelerating CUDA C Applications with Concurrent Streams | |
| – Fundamentals of Accelerated Computing with CUDA C/C++ | |
| – Scaling Workloads Across Multiple GPUs with CUDA C++ | |
| ◦ C++ / Data Structures | Coursera |
| – Object Oriented Data Structures in C++ | |
| ◦ Java | Udemy |
| – Java Spring Boot: Professional eCommerce Project Masterclass | |